STM32WL55JC1

Primo progetto bare-metal: accensione LED verde su PB9

Cortex-M4 • registri memory-mapped • startup minimale • linker script • GCC ARM Embedded

Obiettivo: comprendere l'intera catena dal reset della CPU al pin fisico, senza HAL e senza RTOS.

1. Obiettivo del progetto

Il progetto realizza il più semplice firmware bare-metal utile a comprendere l'avvio e il controllo di una periferica su STM32: dopo il reset della CPU1 Cortex-M4, il firmware abilita il clock di GPIOB, configura PB9 come uscita e porta PB9 a livello alto, accendendo il LED utente verde della scheda.

- Scheda: NUCLEO-WL55JC1.
- MCU: STM32WL55JCI7, famiglia STM32WL5x.
- Core utilizzato in questo esercizio: CPU1 Cortex-M4.
- LED verde utente: PB9, Active HIGH.
- Approccio: accesso diretto ai registri, senza STM32 HAL e senza Zephyr.
- Programmazione/debug hardware tramite ST-LINK-V3E integrato e SWD.

2. Toolchain utilizzata

Componente	Uso
arm-none-eabi-gcc	Compilazione/assemblaggio/link del firmware Cortex-M4
arm-none-eabi-objcopy	Conversione ELF → BIN
arm-none-eabi-size	Dimensione delle sezioni del firmware
arm-none-eabi-objdump	Ispezione sezioni, vector table e disassembly
STM32CubeProgrammer	Connessione ST-LINK, programmazione Flash e verifica
ST-LINK-V3E	Interfaccia integrata tra Mac e MCU via SWD

3. Struttura del progetto

```
STM32-WL55JC/
├── Makefile
├── linker.ld
├── firmware.elf
├── firmware.bin
├── firmware.map
└── src/
    ├── main.c
    └── startup.s
```

I file ELF/BIN/MAP sono generati dalla build; i tre elementi fondamentali scritti manualmente sono main.c, startup.s e linker.ld.

4. Registri e indirizzi ricavati

Elemento	Indirizzo / bit	Funzione
RCC base	0x58000000	Base del blocco RCC
RCC_AHB2ENR	0x5800004C	Abilitazione clock periferiche AHB2
GPIOBEN	bit 1	Abilita il clock di GPIOB per CPU1
GPIOB_MODER	0x48000400	Modalità dei pin GPIOB

PB9 MODE9	bit 19:18	01 = General-purpose output
GPIOB_ODR	0x48000414	Output Data Register
PB9 OD9	bit 9	1 = PB9 HIGH → LED verde ON

Nota: il blocco RCC è mappato nella regione AHB3, mentre RCC_AHB2ENR controlla il clock delle periferiche sul bus AHB2. Le due nozioni non vanno confuse.

5. Memory map usata

Regione	Intervallo	Dimensione usata
User Flash	0x08000000 – 0x0803FFFF	256 KB
SRAM1	0x20000000 – 0x20007FFF	32 KB
SRAM2	0x20008000 – 0x2000FFFF	32 KB

SRAM1 e SRAM2 sono contigue nell'intervallo considerato e, per questo esercizio minimale, vengono descritte al linker come una regione RAM da 64 KB. Il valore iniziale dell'MSP è 0x20010000, cioè il primo indirizzo successivo alla RAM; lo stack cresce verso indirizzi più bassi.

6. main.c

```
#include <stdint.h>

#define RCC_AHB2ENR   (*(volatile uint32_t *)0x5800004C)
#define GPIOB_MODER   (*(volatile uint32_t *)0x48000400)
#define GPIOB_ODR     (*(volatile uint32_t *)0x48000414)

int main(void)
{
    // Enable GPIOB peripheral clock
    RCC_AHB2ENR |= (1U << 1);

    // Read-back: ensure GPIOB clock enable has taken effect
    (void)RCC_AHB2ENR;

    // Configure PB9 as general-purpose output (MODE9 = 01)
    GPIOB_MODER &= ~(3U << 18);
    GPIOB_MODER |=  (1U << 18);

    // PB9 HIGH -> Green LED ON
    GPIOB_ODR |= (1U << 9);

    while (1)
    {
    }
}
```

Le macro trasformano indirizzi assoluti in lvalue volatile a 32 bit. volatile è essenziale perché gli accessi rappresentano operazioni hardware reali e non devono essere eliminati come letture/scritture apparentemente ridondanti.

Le operazioni |= e &= sono read-modify-write: il valore corrente del registro viene letto, modificato e riscritto. Il read-back di RCC_AHB2ENR introduce una lettura effettiva dopo l'abilitazione del clock prima dell'accesso ai registri GPIO.

7. Startup minimale

```
.syntax unified
.cpu cortex-m4
.thumb

.section .isr_vector, "a", %progbits
.word 0x20010000
.word Reset_Handler

.section .text.Reset_Handler
.global Reset_Handler
.type Reset_Handler, %function

Reset_Handler:
    bl main

Loop:
    b Loop
```

Al reset il Cortex-M4 non cerca direttamente main(). I primi due word della vector table forniscono il valore iniziale dell'MSP e il vettore di reset. Reset_Handler esegue quindi un Branch with Link verso main(). Questo startup è intenzionalmente minimale e non inizializza ancora .data e .bss.

8. Linker script

```
ENTRY(Reset_Handler)

MEMORY
{
    FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 256K
    RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 64K
}

SECTIONS
{
    .isr_vector :
    {
```

```
        KEEP(*(.isr_vector))
    } > FLASH

    .text :
    {
        *(.text*)
        *(.rodata*)
    } > FLASH

    .data :
    {
        *(.data*)
    } > RAM AT > FLASH

    .bss :
    {
        *(.bss*)
        *(COMMON)
    } > RAM
}
```

La vector table viene collocata per prima nella Flash, quindi a 0x08000000. .text e .rodata seguono in Flash. .data e .bss sono predisposte in RAM, ma il Reset_Handler attuale non effettua ancora copia/azzeramento: questa sarà un'estensione del progetto.

9. Makefile e build

```
TARGET = firmware

CC      = arm-none-eabi-gcc
OBJCOPY = arm-none-eabi-objcopy
SIZE    = arm-none-eabi-size

CPUFLAGS = -mcpu=cortex-m4 -mthumb

CFLAGS = $(CPUFLAGS) \
         -Wall -Wextra \
         -O0 \
         -g3 \
         -ffreestanding

LDFLAGS = $(CPUFLAGS) \
          -T linker.ld \
          -nostdlib \
          -Wl,-Map=$(TARGET).map
```

```
SOURCES = src/startup.s src/main.c

all: $(TARGET).elf $(TARGET).bin

$(TARGET).elf: $(SOURCES) linker.ld
    $(CC) $(CFLAGS) $(SOURCES) $(LDFLAGS) -o $@
    $(SIZE) $@

$(TARGET).bin: $(TARGET).elf
    $(OBJCOPY) -O binary $< $@

clean:
    rm -f $(TARGET).elf $(TARGET).bin $(TARGET).map
```

Comando di build:

```
make
```

Build osservata prima del read-back RCC:

```
text    data    bss    dec    hex
84      0       0     84     54
```

-O0 mantiene il codice facilmente leggibile durante il debug; -g3 genera informazioni di debug; -ffreestanding segnala un ambiente senza OS/runtime hosted; -nostdlib evita l'inserimento automatico dello startup e delle librerie standard.

10. Verifica dell'ELF

Con:

```
arm-none-eabi-objdump -h firmware.elf
```

è stata verificata la disposizione:

```
.isr_vector  VMA 0x08000000  size 0x08
.text        VMA 0x08000008  size 0x4C
```

La vector table è stata poi letta con:

```
arm-none-eabi-objdump -s -j .isr_vector firmware.elf
```

Output osservato:

```
08000000  00000120 09000008
```

Essendo little-endian, i primi quattro byte rappresentano 0x20010000 (MSP iniziale). I successivi rappresentano 0x08000009: l'handler è fisicamente a 0x08000008 e il bit 0 impostato identifica correttamente lo stato Thumb.

11. Disassembly: dal C alle istruzioni ARM

Il comando:

```
arm-none-eabi-objdump -d -S firmware.elf
```

ha mostrato il percorso reale di esecuzione. Esempi significativi:

Codice C	Istruzione/effetto osservato
RCC_AHB2ENR |= (1U << 1)	ORR con #2 e store a 0x5800004C
GPIOB_MODER &= ~(3U << 18)	BIC con 0x000C0000
GPIOB_MODER |= (1U << 18)	ORR con 0x00040000
GPIOB_ODR |= (1U << 9)	ORR con 512 / 0x200

Il literal pool del codice conteneva esattamente gli indirizzi 0x5800004C, 0x48000400 e 0x48000414, confermando che il compilatore stava effettuando gli accessi memory-mapped progettati.

12. Programmazione della scheda

firmware.elf è stato caricato con STM32CubeProgrammer tramite ST-LINK-V3E integrato. Sono state abilitate la verifica della programmazione e l'esecuzione dopo il download.

```
Download verified successfully
RUNNING Program...
Address: 0x08000000
Application is running
Start operation achieved successfully
```

Risultato hardware: il LED verde collegato a PB9 si è acceso correttamente.

13. Catena completa verificata

```
Reset Cortex-M4
      ↓
Vector table @ 0x08000000
      ↓
MSP = 0x20010000
      ↓
Reset_Handler
      ↓
main()
      ↓
RCC_AHB2ENR.GPIOBEN = 1
      ↓
Clock GPIOB abilitato
      ↓
GPIOB_MODER.MODE9 = 01
      ↓
PB9 configurato come output
      ↓
GPIOB_ODR.OD9 = 1
      ↓
PB9 = HIGH
```

```
      ↓
LED verde ON
```

14. Cosa è stato appreso

- Come il Cortex-M4 entra nel firmware tramite vector table e Reset_Handler.
- Perché stack pointer e memoria SRAM fanno parte del processo di boot.
- Come il linker decide gli indirizzi di vector table, codice e dati.
- Come un registro hardware è esposto come memoria e viene manipolato dal C.
- Come RCC abilita il clock prima dell'uso di GPIOB.
- Come MODER seleziona la funzione del pin e ODR ne controlla il livello logico.
- Come verificare un ELF con objdump prima di programmarlo.
- Come collegare una riga C alle istruzioni ARM Thumb realmente eseguite.
- Come programmare e verificare il firmware tramite ST-LINK/SWD.

15. Limiti intenzionali e prossimi passi

Questo non è ancora uno startup completo da produzione. In particolare, .data non viene copiata dalla Flash alla RAM e .bss non viene azzerata; la vector table contiene solo MSP e Reset_Handler; non sono presenti fault/interrupt handler; il controllo del LED usa ODR con read-modify-write anziché l'accesso atomico tramite BSRR. Sono limitazioni intenzionali, utili a mantenere visibile la meccanica fondamentale.

Prossimo passo pianificato: collegare arm-none-eabi-gdb allo ST-LINK GDB Server, fermarsi a main() e osservare dal vivo PC, SP e i registri RCC_AHB2ENR, GPIOB_MODER e GPIOB_ODR. Successivamente il progetto potrà evolvere verso startup completo, BSRR, blinking/delay, timer, interrupt/NVIC, UART e altre periferiche.

16. Riferimenti tecnici usati durante il progetto

- ST RM0453 Rev. 6 (October 2024) — STM32WL5x Reference Manual: memory map, RCC e GPIO.
- Documentazione Nucleo-WL55JC consultata per la mappatura dei LED: PB9 Green LED, Active HIGH.
- STM32CubeProgrammer per programmazione e verifica tramite ST-LINK.

Stato del progetto: BUILD OK • ELF verificato • FLASH OK • LED verde ON